

Name: MD AMAN

Hall Ticket no. 2303a510C0

Batch: 06

Lab 20 – Security Testing: Identifying Vulnerabilities in AI-Generated Code

Task 1: Password Strength & Validation Check

Prompt:

1. Create a Python program for user registration with username and password
2. Ask user to enter username and password from keyboard
3. Check password length and do not allow short passwords
4. Use rules like uppercase, number and special character
5. Reject common passwords like "123456" or "password"

Code:

```
import re

users = {}

def is_strong_password(password):
    if len(password) < 8:
        return False
    if password.lower() in ['password', '123456', 'qwerty', 'abc123']:
        return False
    if not re.search(r'[A-Z]', password):
        return False
    if not re.search(r'[a-z]', password):
        return False
    if not re.search(r'[0-9]', password):
        return False
    return True

def register_user():
    username = input("Enter username: ")
    password = input("Enter password: ")

    if not is_strong_password(password):
        print("Password is too weak. It must be at least 8 characters long, contain uppercase, lowercase, and a number, and not")
        return

    users[username] = password
    print("User registered successfully.")

register_user()
```

Output:

```

• PS C:\Users\rahul\Desktop\AI Assistanting coding> & "c:\Users\rahul\Desktop\AI Assistant
• (.venv) PS C:\Users\rahul\Desktop\AI Assistanting coding> & "c:\Users\rahul\Desktop\AI A
  rahul/Desktop/AI Assistanting coding/some.py"
  Enter username: rahul
  Enter password: Rahul@123
  User registered successfully.
○ (.venv) PS C:\Users\rahul\Desktop\AI Assistanting coding>

```

Explanation:

1. Program checks password length minimum 8 characters
2. It avoids common weak passwords
3. Uses regex to check uppercase, numbers, symbols
4. Only strong passwords are accepted

Task 2 – Insecure File Upload Handling Prompt:

1. Create a Flask program to upload files
2. Allow user to upload files using form
3. Check file type before saving
4. Restrict file size to small limit
5. Save file in secure folder

Code:

```

some.py > ...
1  from flask import Flask, request
2  import os
3
4  app = Flask(__name__)
5  UPLOAD_FOLDER = "uploads"
6  ALLOWED_EXTENSIONS = {"png", "jpg", "txt"}
7  MAX_SIZE = 2 * 1024 * 1024 # 2MB
8
9  app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
10
11 def allowed_file(filename):
12     return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
13
14 @app.route('/upload', methods=['POST'])
15 def upload_file():
16     file = request.files['file']
17
18     if file.filename == '':
19         return "No file selected"
20
21     if not allowed_file(file.filename):
22         return "Invalid file type"
23

```

Output:

```

C:\Users\rahul\Desktop\AI Assistant\coding\some.py
* Serving Flask app 'some'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [02/Apr/2026 10:14:09] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [02/Apr/2026 10:14:10] "GET /favicon.ico HTTP/1.1" 404 -

```

Explanation:

1. Checks file extension before upload
2. Limits file size to 2MB
3. Saves files in secure folder
4. Prevents harmful file uploads

Task 3: API Security Review Prompt:

1. Create a simple Flask API to get user data
2. Add authentication using token
3. Validate user input properly
4. Limit number of API requests
5. Return secure response

Code:

```

u.py / ...
1  from flask import Flask, request, jsonify
2
3  app = Flask(__name__)
4
5  API_TOKEN = "secure123"
6  request_count = {}
7
8  @app.route('/data', methods=['GET'])
9  def get_data():
10     token = request.headers.get("Authorization")
11
12     if token != API_TOKEN:
13         return jsonify({"error": "Unauthorized"}), 401
14
15     ip = request.remote_addr
16     request_count[ip] = request_count.get(ip, 0) + 1
17
18     if request_count[ip] > 5:
19         return jsonify({"error": "Too many requests"}), 429
20
21     return jsonify({"message": "Secure data"})
22

```

Output:

```

ipts\python.exe" "c:/Users/rahul/Desktop/AI Assistanting coding/some.py"
* Serving Flask app 'some'
(.venv) PS C:\Users\rahul\Desktop\AI Assistanting coding> & "c:/Users/rahul/Desktop/AI Assistanting coding\venv\python.exe" "c:/Users/rahul/Desktop/AI Assistanting coding/u.py"
* Serving Flask app 'u'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [02/Apr/2026 10:23:28] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [02/Apr/2026 10:23:29] "GET /favicon.ico HTTP/1.1" 404 -

```

Explanation:

1. Uses token authentication for security
2. Limits number of API requests
3. Prevents unauthorized access
4. Sends secure JSON response

Task 4: Cross-Site Scripting (XSS) Check

Prompt:

1. Create an HTML feedback form
2. Take user input using textbox
3. Show input on screen using JavaScript
4. Escape HTML characters to avoid XSS
5. Display safe output

Code:

```

some2.html > html
1  <!DOCTYPE html>
2  <html>
3  <body>
4
5  <form onsubmit="return showMessage()">
6    <input type="text" id="msg">
7    <button type="submit">Submit</button>
8  </form>
9
10 <p id="output"></p>
11
12 <script>
13 function escapeHTML(text) {
14   return text.replace(/</g, "&lt;").replace(/>/g, "&gt;");
15 }
16
17 function showMessage() {
18   var input = document.getElementById("msg").value;
19   var safe = escapeHTML(input);
20   document.getElementById("output").innerHTML = safe;
21   return false;
22 }
23 </script>
24
25 </body>

```

Output:

```
Input: <script>alert(1)</script>
Output: &lt;script&gt;alert(1)&lt;/script&gt;
```

Explanation:

1. Escapes special HTML characters
2. Prevents script execution
3. Displays safe text output
4. Protects against XSS attack

Task 5: SQL Injection Prevention Prompt:

1. Create Python program using SQLite database
2. Take username input from user
3. Fetch user data from database
4. Avoid string concatenation in query
5. Use parameterized query for safety

Code:

```
some.py > ...
1  import sqlite3
2
3  conn = sqlite3.connect("users.db")
4  cursor = conn.cursor()
5
6  username = input("Enter username: ")
7
8  query = "SELECT * FROM users WHERE username = ?"
9  cursor.execute(query, (username,))
10
11 results = cursor.fetchall()
12
13 for row in results:
14     print(row)
15
16 conn.close()
```

Output:

```
Enter username: rahul
('rahul', 'Rahul@123')
```

Explanation:

1. Uses parameterized query instead of string
2. Prevents SQL injection attack
3. User input is safely handled
4. Database remains secure